Programming Project / C++

Auto Rental Fleet Management System

Student 1: Simion Sirb

Student 2: Csiszer Mihai

I. Task Description

This project implements a **car rental fleet management system** in C++. The system consists of two separate executables that share data through text files. Each application presents a fully **interactive terminal menu** navigated with the keyboard arrow keys – no command-line arguments or std::cin are used.

Student 1 – Fleet Manager (app_1)

- Adding, deleting, and modifying vehicles in the fleet
- Searching for a vehicle by brand name
- Viewing all vehicles with status
- Viewing all existing reservations

Student 2 – Customer Interaction (app_2)

- Viewing vehicles currently available for rental
- Making a reservation for a specified date range
- Cancelling an existing reservation
- Viewing all reservations belonging to a customer ID

II. Data Structures Used by the Team

The following classes are used. **Car** inherits from the abstract **Vehicle** base (*inheritance*). **Reservation** owns a **Car** object directly (*composition*) – a reservation cannot exist without a vehicle.

Class	Fields / Notes

Vehicle (abstract base)	`string id`
	`string brand`
	`string vehicleType`
	`bool available`
	`int seats`
	
	`Pure virtual: toFileLine(), display()`

Car
(extends Vehicle)
	`string fuelType`
	`double pricePerDay`
	
	`Inherits all Vehicle fields.`
	`Adds: fromFileLine() static factory`

Date	`int day, month, year`
	
	`Methods: toString(), isValid(), operator<, operator<=`

Reservation	`string reservationId`
	`Car car  ← composition`
	`Date startDate, endDate`
	`string customerId`

III. File Structure

The two applications share three plain-text files in the working directory. Both executables reload files before every operation to stay in sync.

flota.txt

Stores all vehicles in the rental fleet:

```
<number of vehicles>
<id> <brand> <vehicleType> <available> <seats> <fuelType> <pricePerDay>
...
```

rezervari.txt

One reservation per line:

```
<reservationId> <customerId> <carId> <startDay> <startMonth> <startYear>
<endDay> <endMonth> <endYear>
...
```

clienti.txt

Maps customer IDs to display names:

```
<customerId> <customerName>
...
```

IV. Interactive Menu Navigation

Both applications use **ncurses** to render a full-screen, arrow-key navigable menu. No command-line arguments are required – the user simply launches the binary and uses the keyboard to navigate.

Key	Action
↑ / ↓	Move the highlight up or down through menu options

ENTER	Confirm the highlighted option
Q or ESC	Go back to the previous menu / quit the application

V. Menu Screens per Application

Application 1 – Fleet Manager (./app_1)

Menu Option	Sub-actions / Prompted Fields
View all vehicles	Displays ID, brand, type, seats, fuel, price, availability
Add vehicle	Prompts: Brand, Type, Seats, Fuel type, Price/day
Delete vehicle	Prompts: Vehicle ID – removes entry from flota.txt
Modify vehicle	Prompts: Vehicle ID, then sub-menu: Price / Availability / Seats / Fuel type / Brand
Search vehicle by brand	Prompts: Brand keyword – case-insensitive substring match
View all reservations	Reads rezervari.txt and displays all bookings
Quit	Exits the application

Application 2 – Customer Interaction (./app_2)

Menu Option	Sub-actions / Prompted Fields
View available vehicles	Lists only vehicles where available = true
Make a reservation	Prompts: Customer ID, Car ID, Start date (d/m/y), End date (d/m/y). Validates dates; marks car unavailable; writes to rezervari.txt
Cancel a reservation	Prompts: Reservation ID – removes entry, sets car back to available
View my reservations	Prompts: Customer ID – filters and lists matching reservations
Quit	Exits the application

Build & Run: `make seed && make all` then `./app_1` or `./app_2`. Requires **libncurses** (apt install libncurses5-dev). No std::cin is used anywhere. OOP relationships: **Car** inherits **Vehicle** (inheritance) and **Reservation** contains a **Car** (composition).